

Data Structures in Python

Simple Guide to Storing and Organizing Data

This document explains the basic data structures in python such as List, Tuples, Set, Frozen Set, Dictionary.

Presented by

Madhivanan M

Data Structures in Python

Simple Guide to Storing and Organizing Data

Introduction:

Python provide some built in tools called Data Structures that help us to store, organize and manage data easily.

They are like containers that holds different type of information such as numbers, words, or lists of items.

Each data structures has its own way of storing and handling data:

- Some can change after creation (like Lists).
- Some can't be change (like Tuples).
- Some don't allow duplicates (like Sets and frozen sets).
- And some store data in key: value pairs (like Dictionaries).

Choosing right data structure helps make your python program cleaner, faster and more efficient.

Python Built-in Data Structures

Learn about python's main data containers – List, Tuple, Set, Frozen set, Dictionary and how they help store and organize data efficiently

List: Ordered and changeable.

Tuple: Ordered and unchangeable.

Set: Unordered and unique values

Frozen set: Unordered and unchangeable unique values.

Dictionary: Key – Value pairs

List []

A list is an ordered, mutable(changeable) collection that allows duplicate elements. It can store different data types (Numbers, Strings, Booleans, etc.) in a single container.

Characteristics of list:

- **Ordered:** Elements maintain the order in which they are inserted.
- **Mutable:** you can add, remove and change the element after creating the list.
- **Allows Duplicates:** The same value can appear more than once.
- **Index Based:** Each element has an index (starting from 0).
- **Heterogeneous:** you can store multiple data type in one list.

Syntax:

```
List_name = [element1, element2, element3, .....]
```

Example:

```
fruits = ["apple", "banana", "orange"]  
print(fruits)
```

Output: ['apple', 'banana', 'orange']

Index-Based Operations:

These operations work using index number (position of the element in the list).

- **Access:** Get element by index →
`fruits[1]`
output: 'banana'
- **Change:** Change an element value →
`fruits[1] = "cherry"`
output: ['apple', 'cherry', 'orange']

- **Delete:** Remove element using index →

del fruits[2]

output: ['apple', 'cherry']

#if you didn't mention the index "**del fruits**" then it will delete the entire list.

- **Slicing:** Access range of elements →

fruits[0:2]

output: ['apple', 'cherry'].

- **Pop:** Remove element by index (returns it) →

removed_item = fruits.pop(0)

print(removed_item)

output: ['apple'].

Print(fruits)

output: ['cherry'].

#if you didn't mention the index inside the **pop()** function, it will remove and return the last element from the list.

Value-Based Operations:

This operations work using the value of the element.

- **Append:** Adds an item to the end →

fruits.append('banana')

output: ['cherry', 'banana'].

- **Insert:** Adds an item at specific position→

fruits.insert(1, 'orange')

output: ['cherry', 'orange', 'banana'].

- **Remove:** Remove first matching value from the list →

fruits.remove('orange')

output: ['cherry', 'banana'].

Other List Operation:

- **Clear:** Removes all element →
`fruits.clear()`
output: []
Example: a = [1,2,3,2,2]; b = [4,5,6]
- **Count:** Count how many times a value appears →
`a.count(2)`
output: 3.
- **Sort:** Sorts list (ascending by default) →
`a.sort()`
output: [1,2,2,2,3].
- **Reverse:** Reverse the list order →
`a.reverse()`
output: [3,2,2,2,1].
- **Extend:** Adds element from another list →
`a.extend(b)`
output: [3,2,2,2,1,4,5,6].
- **Copy:** Makes a copy of the list →
`c = b.copy()`
`print(c)`
output: [4,5,6].
- **Len:** Return number of elements →
`len(c)`
output: 3.

Tuple ()

A tuple is an ordered, immutable (unchangeable) collection of items. Once created, you can't add, remove, or modify its elements. Tuple are often used to store fixed data that should not change during the program.

Characteristics of Tuple:

- Ordered: Element maintain the order in which they are inserted.
- Immutable: can't modify after creation (no- add, remove, update).
- Allows Duplicates: Duplicate values are allowed.
- Index Based: Each element has an index (starting from 0).
- Heterogeneous: you can store multiple data type in one tuple.

Syntax:

```
Tuple_name = (element1, element2, element3, .....).
```

```
Single_tuple = (element,)
```

Example:

```
colour = ("red", "green", "blue").
```

```
print(colour)
```

Output: ('red', 'green', 'blue', 'red')

Index-Based Operations:

Even though tuples are immutable, you can access elements by index perform read only operations.

- **Access:** Get element by index →
`colour[1]`
output: 'green'.
- **Slicing:** Get a part of the tuple →
`colour[0:2]`
output: ('red', 'green').
- **Index:** Find position of value →
`colour.index("blue")`
output: 2.

Other Tuple Operations:

- **Count:** Count how many times a value appears →
`colour.count('red')`
output: 2.
- **Length:** Get total number of elements →
`len(colour)`
output: 4.

Set {}

A set is an unordered, mutable collection of unique elements. It is mainly used when you want to store data without duplicates, perform fast mathematical operations like union, intersection and difference.

Characteristics of set:

- **Unordered:** Element do not have a fixed position or index.
- **Mutable:** you can add or remove elements after creation.
- **No Duplicates:** Each value appears only once.
- **Un-indexed:** you can't access elements using index or slicing.
- **Heterogeneous:** you can store different data type.

Syntax:

```
Set_name = {element1, element2, element3, ....}
```

Example:

```
fruits = {"apple", "orange", "banana", "apple"}
```

```
print(fruits)
```

Output: {"orange", "banana", "apple"}

Duplicate element apple is automatically removed.

Value-Based Operations:

- **Add:** Add one element →
fruits.add("mango")
 output: {'orange', 'banana', 'mango', 'apple'}.
- **Update:** Adds multiple elements →
fruits.update(["cherry", "kiwi"])
 output: {'orange', 'apple', 'kiwi', 'banana', 'mango'}.

- **Remove:** Remove element →
`fruits.remove("kiwi")`
 output: {'apple', 'banana', 'mango', 'orange'}.
- **Discard:** Remove element (no error – if value not found) →
`fruits.discard("mango")` →
 output: {'apple', 'banana', 'orange'}.
- **Pop:** Removes and returns a random element →
`fruits.pop()`
 output: {'apple', 'banana'}.
- **Clear:** Remove all element in the set →
`fruits.clear()`
 output: {}.

Mathematical Set Operations:

Example: $a = \{1,2,3\}$; $b = \{3,4,5\}$

- **Union:** Combines all unique elements →
`a.union(b)`
 output: {1,2,3,4,5}.
- **Intersection:** Common elements in both sets →
`a.intersection(b)`
 output: {3}.
- **Difference:** Element in the 1st set that are not common to both sets →
`a.difference(b)`
 output: {1,2}.
- **Symmetric Difference:** Element not common to both →
`a.symmetric_difference(b)` →
 output: {1,2,4,5}.

Frozen Set - frozenset()

A Frozen set is an immutable version of a set. Once created you can't add, remove or change its elements. It is useful when you need the set can't be modified. Such as using it as a key in a dictionary or storing it inside another set.

In python, dictionary keys must be unchangeable(immutable). so, you can't use a normal set as a key because sets can be changed.

Also, a normal set can only store immutable items inside it. So, you can't put a normal set inside another set but a frozenset can do both operations.

Characteristics of frozenset:

- Unordered: Element do not have a fixed position or index.
- Immutable: Can't be changed after creation (no add/remove).
- No Duplicates: Each element appears only once.
- Unindexed: Can't access elements by index.
- Heterogeneous: Can store different data types.
- Hashable: Can be used as a key in dictionary or as an element in another set.

Syntax:

```
Frozenset_name = frozenset(iterable)
```

Example:

```
fruits = frozenset(["apple", "orange", "mango", "kiwi", "apple"])  
print(fruits)
```

Output: frozenset({'apple', 'kiwi', 'mango', 'orange'})

Duplicate element apple is automatically removed.

Mathematical set operations:

Even though frozen sets are immutable, they still support mathematical set operations like union, intersection and difference.

```
a = frozenset(["apple", "orange", "mango"]); b = {"mango", "kiwi", "apple"}
```

- **Union:** Combine element from both →
a.union(b)
 output: frozenset({'apple', 'kiwi', 'mango', 'orange'}).
- **Intersection:** Common elements in both sets →
a.intersection(b)
 output: frozenset({'apple', 'mango'}).
- **Difference:** Elements in 1st but not in 2nd →
a.difference(b)
 output: frozenset({'orange'}).
- **Symmetric Difference:** Elements not common to both →
a.symmetric_difference(b)
 output: frozenset({'kiwi', 'orange'}).

Some Examples:

- Key = frozenset(['id', 'name'])
 Data = {key: "student info"}
 print(data)
 output: {frozenset({'id', 'name'}): 'student info'}
- a = frozenset({1,2,3})
 b = {a}
 output: {frozenset({1,2,3})}

Dictionary {key: value}

A Dictionary is a collection of key-value pairs. Each key in dictionary is unique, and every key is mapped to a specific value. It is used to store data in a structured way, where you can access values using keys instead of indexes.

Characteristics of Dictionary:

- Unordered: Element didn't have a fixed position or index before python 3.7, but dictionaries are ordered in python 3.7 and later.
- Mutable: You can add, remove or update items.
- key-value pair: Each element has a unique key and its associated value.
- Unique keys: Duplicate keys are not allowed.
- Indexed by keys: Access elements using key names, not numeric index.
- Heterogeneous: Can store different data types for keys and values.

Syntax:

```
dict_name = {key1: value1, key2: value2, key3: value3, ...}
```

Example:

```
car = {"brand": "ford", "model": "mustang", "year": 1964}  
print(car)
```

output:

```
{'brand': 'ford', 'model': 'mustang', 'year': 1964}
```

Example1:

```
info = dict(name = "Bob", age = 26) # using dict() constructor
```

Accessing Dictionary Elements:

- **Access by key:** Get the value of a key →
`car["model"]`
 output: 'mustang'.
- **Using get():** Returns values if key exists, else none →
`car.get("year")`
 output: 1964.
- **Keys():** Returns all keys →
`car.keys()`
 output: dict_keys(['brand', 'model', 'year']).
- **Values():** Returns all values →
`car.values()`
 output: dict_values(['ford', 'mustang', 1964]).
- **Items():** Returns all key-value pairs →
`car.items()`
 output: dict_items([('brand', 'ford'), ('model', 'mustang'), ('year', '1964')]).

Value Based Operations:

- **Add and update Item:** Add a new key-value pair →
`car["color"] = "red"`
 output: {'name': 'ford', 'model': 'mustang', 'year': 1964, 'color': 'red'}. # If the key already exists, its value will be updated. If the key does not exist, a new key-value pair will be created.
- **Update Multiple:** Updates multiple keys →
`car.update({"rpm": 4400, "color": "red"})`
 output: {'name': 'ford', 'model': 'mustang', 'year': 1964, 'color': 'red', 'rpm': 4400}.
- **Remove specific item:** Remove a key-value pair →
`car.pop("model")`
 output: {'name': 'ford', 'year': 1964, 'color': 'red', 'rpm': 4400}.

- **Remove last item:** Removes the last key-value pair →
`car.popitem()`
output: {'name': 'ford', 'year': 1964, 'color': 'red'}.
- **Delete key:** Delete a key-value pair →
`del car["year"]`
output: {'name': 'ford', 'color': 'red'}.
del car → the variable car itself is deleted from memory.
- **Clear:** Removes all item →
`car.clear()`
output: {} #empty

Dictionary loop:

```
car = {"brand": "ford", "model": "mustang", "year": 1964}
```

- **for key in car:**
`print(key)`
output: name model year # each one on the next line
- **for value in car.values():**
`print(value)`
output: ford mustang 1964 # each one on the next line
- **for key, value in car.items():**
`print(key)`
`print(value)`
output: name model year # each one on the next line

output: ford mustang 1964 # each one on the next line.

Nested Dictionary:

```
Students = {student1: {"name": "Jack", "age": 25},
            student2: {"name": "Alex", "age": 26}}
```

```
print(student[student2][name])
```

output: Alex.

=====

Thank you